真にスレッドセーフな HashMapとは

2013/11/16
第3回 #渋谷java
せとあずさ♂

- @setoazusa
- #tddbc 横浜(2011～2013)
- #agilesamurai #横浜道場
- 新規開発にお勧めできないOSSのコミッタ
- #yokohamarb
- 最近、MacからWindows8に乗り換えました
- チャンキヨかわいいよチャンキヨ
- miwaは自慢の妹です

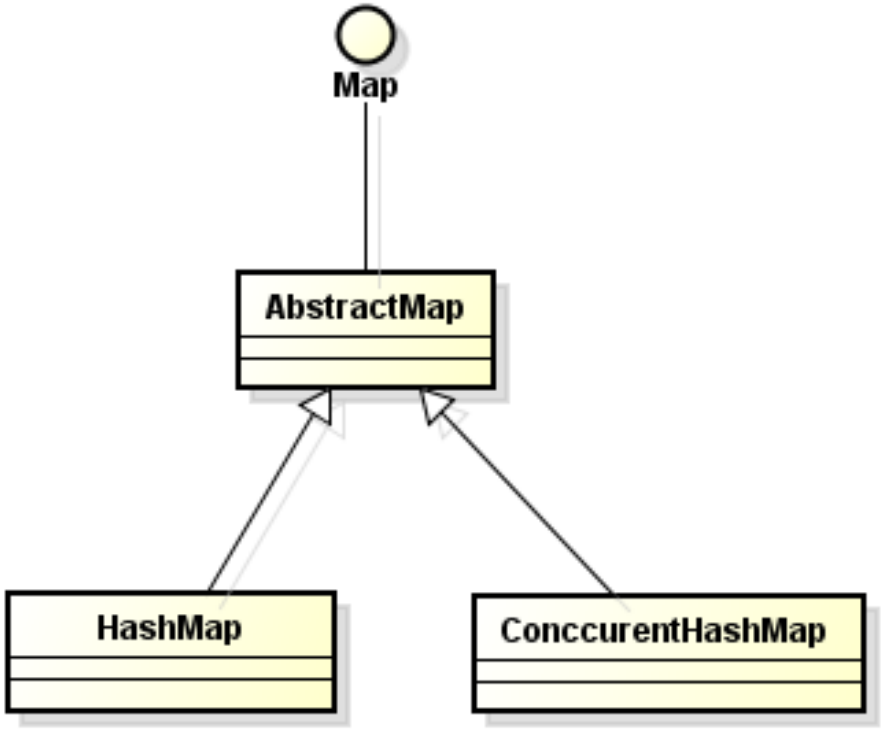
- スレッドセーフとは…?

- スレッドセーフなHashMap?

- ConcurrentHashMap使えばいいんじゃね？

残念！

- ConccurentHashMapはHashMapではありません！



map.put(“key”, null);

HashMap → そのまま通す
ConccurentHashMap → ぬるぽ

これはバグではありません

例外:

`UnsupportedOperationException` - put オペレーションがこのマップでサポートされない場合

`ClassCastException` - 指定されたキーまたは値のクラスが原因で、このマップにそれらを格納できない場合

`NullPointerException` - 指定されたキーまたは値が null であり、このマップが null のキーまたは値を許可しない場合

`IllegalArgumentException` - 指定されたキーまたは値のあるプロパティーが原因で、このマップにそれらを格納できない場合

http://docs.oracle.com/javase/jp/7/api/java/util/Map.html#put%28K,%20V%29

- やむを得ないので、Collections#synchronizedMapを使うわけですが…

- synchronizedMapって、iterator(foreach)を回す場合は呼び出し元で同期する必要があるじゃないですか？

- 呼び出し元でどうやっても同期できないケースが1つだけあります。

- シリアライズ

```java
private void writeObject(java.io.ObjectOutputStream s)
    throws IOException
{
    // Write out the threshold, loadfactor, and any hidden stuff
    s.defaultWriteObject();

    // Write out number of buckets
    if (table==EMPTY_TABLE) {
        s.writeInt(roundUpToPowerOf2(threshold));
    } else {
       s.writeInt(table.length);
    }

    // Write out size (number of Mappings)
    s.writeInt(size);

    // Write out keys and values (alternating)
    if (size > 0) {
        for(Map.Entry<K,V> e : entrySet0()) {
            s.writeObject(e.getKey());
            s.writeObject(e.getValue());
        }
    }
}
```

どんなケースでひっかかったのか

- WebアプリのHttpSessionにHashMapを格納していて、
- Tomcatのセッションクラスタリングがレプリケーションのためにを HashMapをシリアライズして、
- そのシリアライズの最中にリクエストからの処理(シリアライズとは別スレット)がHashMapにput

どうする？

- Javaの言語仕様として、継承できない
- 呼び出し元は 標準APIの中

- スレッドごとに、オブジェクトを分割すればいい

```java
/**
 * {@link java.util.HashMap}のwriteObjectメソッドがスレッドセーフでないため、イ
 * {@link Set}を返しています。
 *
 * @return entrtySet
 * @see java.util.HashMap#entrySet()
 */
@Override
public Set<Map.Entry<K, V>> entrySet() {
        synchronized (delegated) {
                Set<Entry<K, V>> set = new LinkedHashMap<K, V>(delegated)
                                .entrySet();
                return set;
        }

}
```

https://gist.github.com/azusa/7052618

まとめ

- クラスがスレッドセーフかどうかはコンテキストに依存するということを理解しない人は、豆腐の角に頭をぶつけて(ry
- コレクションAPIには地雷があります
- スケールアウトの容易さという点について、railsが羨ましいです
- Java並行処理プログラミング読みましょう